

Atividade 3: Auditoria de Testes de Software e Plano de Evolução da Qualidade

Disciplina: Engenharia de Software (COMP0503) - UFS

Adley Breno de Melo Moraes (Matrícula: 202300026985)

30 de junho de 2026

1 Contextualização e Objetivo

O ecossistema analisado é o Jan ([menloresearch/jan](https://github.com/menloresearch/jan)), um assistente de inteligência artificial de código aberto focado em privacidade, que opera modelos de linguagem (LLMs) de forma 100% offline e local. O objetivo central desta terceira etapa da auditoria é realizar um diagnóstico profundo da estratégia de testes do projeto. Iremos investigar se a validação atual garante a segurança necessária para a evolução contínua da ferramenta, identificando lacunas de cobertura e propondo um plano concreto de melhoria fundamentado em boas práticas de engenharia de software.

2 O Legado: Conexão com as Atividades 1 e 2

Esta análise de testes herda o contexto arquitetural e operacional levantado nas auditorias anteriores, mantendo a rastreabilidade da investigação e direcionando o foco para os pontos de maior risco do sistema:

- **Requisitos Críticos (Atividade 1):** Mapeamos fluxos centrais de negócio, como a integração do adaptador SDK do Gemini, a adição do provedor de nuvem MiniMax e a complexa lógica de controle de limite de tokens. Nesta etapa, é imperativo verificar se essas integrações vitais estão devidamente protegidas por testes automatizados.
- **Dívida Técnica e Riscos Arquiteturais (Atividade 2):** A auditoria forense expôs violações severas de design, destacando a classe `ModelManager` como um "God Object" (violação clara de SRP) e a duplicação massiva de blocos de tratamento de exceção em múltiplos arquivos (quebra do princípio DRY). O diagnóstico avaliará se a suíte de testes atual possui uma malha de segurança robusta o suficiente para permitir a refatoração dessas classes sem causar regressões em produção.
- **Passivos Ocultos:** A identificação de marcações `// TODO` e `// FIXME` em arquivos centrais de captura de exceções nativas (`error.rs`) e delegações no `tauri.ts` levanta sérias suspeitas sobre a cobertura de cenários de falha, aspecto que será diretamente confrontado com a estratégia de validação do Eixo C.

3 Eixos de Investigação (O "Deep Dive")

3.1 Eixo A: Estratégia Atual de Testes

Item 4: Existe integração com CI/CD para executar testes automaticamente em Pull Requests ou pushes?

(a) **Evidência:** A infraestrutura de automação está concentrada no diretório `.github/workflows`. A Figura 1 comprova a existência de pipelines dedicados, com destaque para os arquivos `jan-linter-and-test.yml` e a suíte de qualidade `autoqa-reliability.yml`.

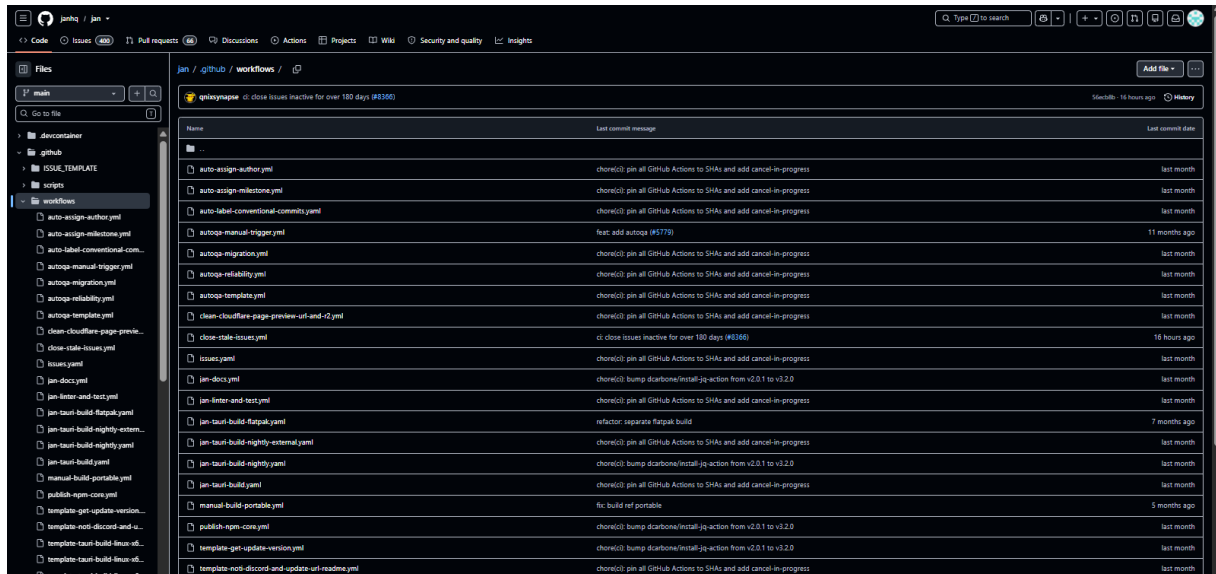


Figura 1: Fluxos de trabalho automatizados via GitHub Actions (CI/CD).

(b) **Diagnóstico:** O ecossistema Jan possui um nível avançado de maturidade operacional em CI/CD. A presença do pipeline `jan-linter-and-test.yml` indica que a equipe bloqueia falhas humanas rodando análises estáticas (linter) e testes de forma acoplada. As rotinas `autoqa` demonstram uma esteira de automação voltada para confiabilidade (reliability) e validação de regressões sem depender exclusivamente de execução manual.

(c) **Risco:** Baixo. A existência desses gatilhos mitiga severamente o risco de inserção de código quebrado na branch principal (main), pois os PRs só avançam se as validações do GitHub Actions passarem.

(d) **Recomendação:** Embora o pipeline execute os testes, recomenda-se configurar o fluxo do `jan-linter-and-test.yml` para gerar e exportar um "Coverage Badge" e um relatório detalhado de cobertura de código como artefato público no próprio PR. Isso dará transparência imediata sobre quais linhas o desenvolvedor esqueceu de testar antes do Code Review.

3.2 Eixo B: Qualidade Técnica dos Testes

Item 2: Os testes são isolados, independentes e reproduzíveis?

(a) **Evidência:** O artefato de validação central localizado dentro do diretório `/tests` do projeto não é uma suíte de código, mas sim um documento estático (`checklist.md`).

A Figura 2 comprova que a equipe utiliza um longo *Release Checklist* estruturado em texto livre (com seções como "Initial update / migration Data check") contendo dezenas de passos para validação humana.

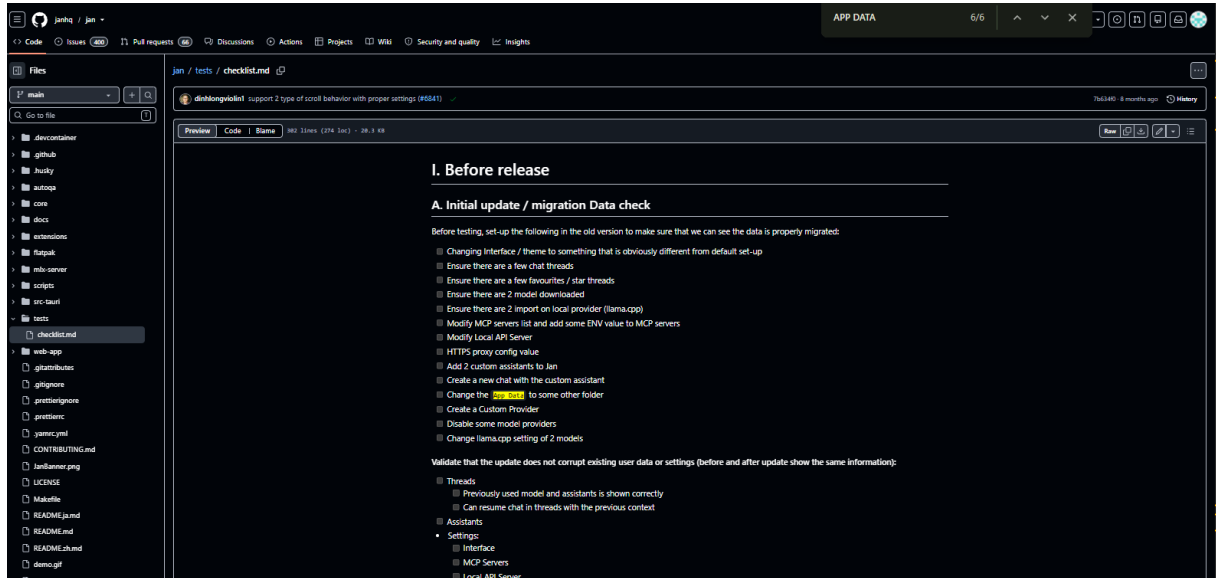


Figura 2: Evidência de validação manual através do arquivo `checklist.md` na pasta `tests`.

(b) **Diagnóstico:** A qualidade técnica da suíte primária diverge frontalmente das práticas modernas de engenharia de software automatizada. A dependência desse *checklist* demonstra que os testes de ponta a ponta (E2E) não são isolados em código, não utilizam *mocks* e dependem inteiramente do estado global da máquina do testador humano, exigindo interações braçais repetitivas para aprovar um release.

(c) **Risco:** Alto. Essa estratégia fere o princípio da reprodutibilidade. Testes manuais não escalam à medida que o ecossistema aumenta, sendo frágeis e altamente suscetíveis à fadiga humana, o que eleva exponencialmente a chance de regressões passarem despercebidas.

(d) **Recomendação:** A equipe deve investir na migração progressiva dos fluxos mapeados no `checklist.md` para testes de interface e comportamento (E2E) automatizados utilizando *frameworks* modernos (ex: Cypress, Playwright), garantindo isolamento de estado a cada execução no CI.

3.3 Eixo C: Lacunas, Riscos e Falhas Potenciais

Item 2: Quais módulos (com dívida técnica apontada na A2) apresentam maior risco caso sejam alterados?

(a) **Evidência:** A auditoria forense da Atividade 2 apontou a classe `ModelManager` como um crítico "God Object". A varredura por testes cobrindo este módulo (Figura 3) revela que não existe um arquivo `ModelManager.test.ts`. A única menção relevante ocorre no arquivo `ExtensionProvider.test.tsx`, onde o `ModelManager` é injetado como um *mock* vazio genérico (`vi.mock(..., ModelManager: class {})`).

(b) **Diagnóstico:** O ecossistema carece de proteção contra regressões em seu núcleo arquitetural mais vulnerável. O fato de os desenvolvedores precisarem *mockar* o módulo

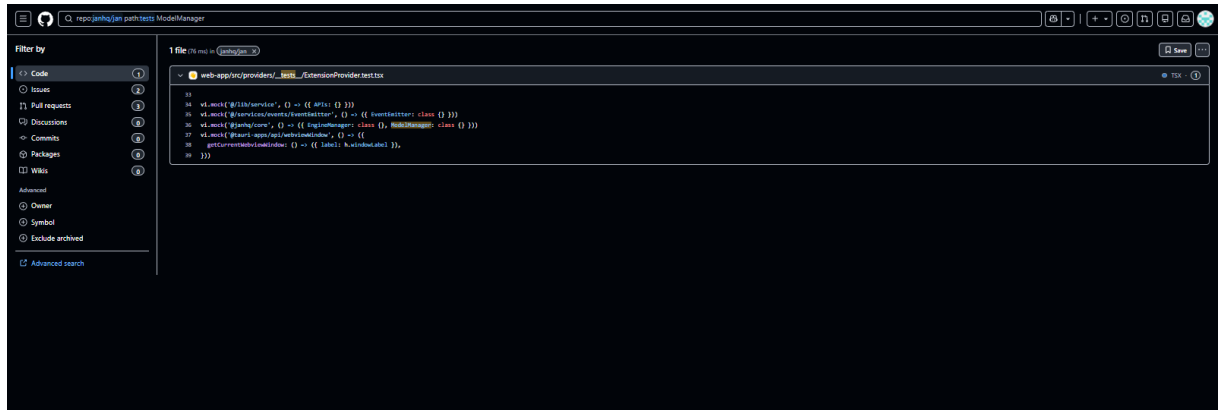


Figura 3: Evidência da ausência de testes unitários reais para o God Object **ModelManager**.

inteiro com uma classe vazia apenas para testar componentes de UI periféricos comprova o alto acoplamento. A lógica interna e real do **ModelManager** está completamente descoberta e sem validação isolada.

(c) **Risco:** Crítico (Alto). Sem testes unitários, qualquer tentativa futura de aplicar os padrões de refatoração sugeridos (como o *Chain of Responsibility* mapeado na A2) será um voo às cegas. Alterações estruturais na camada de gerenciamento podem causar falhas silenciosas na comunicação com as APIs de IA.

(d) **Recomendação:** Antes de iniciar qualquer esforço de refatoração estrutural (desacoplamento via SOLID), a equipe deve aplicar a técnica de "Caracterização de Testes", criando o arquivo **ModelManager.test.ts** para envolver o código legado em testes de unidade e congelar seu comportamento atual, garantindo uma rede de segurança para a alteração arquitetural.

4 Plano de Evolução da Qualidade

Com base na auditoria arquitetural (Atividade 2) e no diagnóstico de testes, propõe-se um plano de evolução estruturado para elevar a maturidade do projeto Jan:

1. Priorização:

- **Prioridade 1:** Criar uma malha de testes unitários de caracterização para a classe **ModelManager** e componentes inflados do Tauri, viabilizando uma refatoração segura (SOLID).
- **Prioridade 2:** Automatizar o longo *Release Checklist* manual (atualmente em `checklist.md`) transformando-o em rotinas de teste E2E.
- **Prioridade 3:** Isolar o tratamento de exceções (atualmente duplicado - violação DRY) em um interceptador de rede testável.

2. **Camadas de Foco:** O investimento inicial será focado na **Camada de Unidade** (para blindar a lógica de negócios e as regras do **CoreSDK**) e na **Camada de Interface/E2E** (para reduzir a fadiga de testes braçais a cada *release*).

3. **Implantação Gradual:** A implantação seguirá a estratégia de "Caracterização". Nenhum desenvolvedor deve reescrever o código de produção agora; o primeiro passo

é escrever testes que apenas "congelem" o comportamento legado atual. Só após essa blindagem o time de arquitetura aplicará padrões como *Chain of Responsibility*.

4. Automação:

- **Ferramentas:** Adoção do *Playwright* ou *Cypress* para varrer a interface de forma autônoma simulando as ações do *Release Checklist*.
 - **Pipeline:** Integrar o *Codecov* na suíte `jan-linter-and-test.yml` do GitHub Actions com regras de bloqueio (*Status Check*) que impeçam o *merge* de Pull Requests que diminuam a cobertura total do código.
5. **Critérios de Sucesso:** O avanço de maturidade será validado por dois critérios: (1) Eliminação da necessidade humana para rodar as rotinas básicas do *checklist* de regressão, e (2) Exibição pública de *badges* de cobertura (Cobertura $\geq$ 80%) validados por integração contínua na *branch* principal.

5 Contribuição dos Integrantes

- **Adley Breno de Melo Moraes:** 100% (Execução individual da análise, documentação e auditoria).